Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 22.Б15-мм

Расширение функциональности системы профилирования датасетов в приложении Desbordante

ШАЛЬНЕВ Владислав Александрович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
ассистент кафедры ИАС Г. А. Чернышев

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **4**

1.1. Цель работы 4

1.2. Требования к функциональности 4

2. Обзор **5**

2.1. Обзор примитивов 5

2.2. Существующая реализация 10

2.3. Обновление статуса и результата задачи 14

3. Описание решения **16**

3.1. Модели базы данных 16

3.2. Загрузка и предобработка датасетов 17

3.3. Архитектура примитивов 19

3.4. Сохранение статуса и результатов задач 21

3.5. Получение результатов 22

4. Развертывание и тестирование **25**

4.1. Развертывание 25

4.2. Тестирование 25

5. Заключение **26**

Список литературы **27**

Введение

Рост объёмов данных в науке и бизнесе сопровождается развитием инструментов для их анализа. Такие системы позволяют исследовать структуру данных и выявлять закономерности. К числу подобных инструментов относится профилирование данных, позволяющее обнаруживать зависимости, ограничения и иные свойства датасетов. Открытый профилировщик данных Desbordante [2] предоставляет пользователю возможность исследовать различные типы датасетов с помощью библиотеки для языка Python или веб-приложения, в котором серверная часть выполняет задачи профилирования.

После реимплементации [9] серверной части веб-приложения система профилирования датасетов имела ограниченную функциональность по сравнению с Python-библиотекой: поддерживались только два примитива — функциональные зависимости (`FD`) и приближённые функциональные зависимости (`AFD`), каждая задача работала с одним датасетом, а результаты хранились целиком без пагинации.

Это ограничивало возможности пользователей веб-приложения: они не могли применять примитивы, доступные в библиотеке, — такие как метрические зависимости (`MD`), ассоциативные правила (`AR`), алгебраические ограничения (`AC`) и другие. Кроме того, отсутствовала поддержка задач, требующих нескольких датасетов, а хранение результатов целиком без пагинации создавало проблемы при работе с большими объёмами данных.

В рамках данной работы была спроектирована и реализована архитектура примитивов профилирования данных, реализованы 12 примитивов, добавлена поддержка нескольких датасетов в одной задаче, а также реализовано пагинированное хранение и получение результатов задач. Реализованное решение направлено на расширение функциональности системы профилирования датасетов в веб-приложении Desbordante и создание основы для дальнейшего добавления новых примитивов.

1. Постановка задачи

1.1. Цель работы

Целью работы является расширение функциональности системы профилирования датасетов в веб-приложении Desbordante. Для достижения цели были поставлены следующие задачи:

1. провести обзор существующей реализации задач профилирования в серверной части Desbordante;
2. провести обзор требуемых примитивов: параметры алгоритмов, работа с датасетами, возвращаемый результат;
3. спроектировать и реализовать обновлённую систему профилирования, добавить поддержку требуемых примитивов;
4. реализовать хранение и получение результатов задач;
5. развернуть [10] и протестировать новую функциональность.

1.2. Требования к функциональности

К реализуемой в рамках данной работы функциональности были выдвинуты следующие требования:

1. реализовать поддержку следующих примитивов: `FD`, `AFD`, `FD_VERIFICATION`, `AFD_VERIFICATION`, `AC`, `ADC`, `AR`, `DD`, `MD`, `MFD_VERIFICATION`, `NAR`, `PFD`;
2. каждый примитив должен поддерживать соответствующий тип датасетов (табличный, транзакционный или граф);
3. получение результатов задачи и списка задач должно поддерживать постраничный вывод с фильтрацией и сортировкой.

2. Обзор

2.1. Обзор примитивов

Цель данного обзора — изучить требуемые примитивы профилирования: с какими типами датасетов они работают, какие параметры конфигурации принимают, какие алгоритмы доступны и каковы их значения по умолчанию, что возвращается в результате. Эта информация необходима для разработки архитектуры поддержки примитивов в серверной части Desbordante.

В качестве источников использовались два репозитория. Первый — `desbordante-stubs` [12]: репозиторий с Python-стабами (type stubs), сгенерированными инструментом `pybind11-stubgen` из C++-биндингов библиотеки Desbordante [2]. Поскольку ядро Desbordante реализовано на C++, сигнатуры классов и методов не документированы средствами Python; стабы устраняют этот пробел, предоставляя типизированные сигнатуры, доступные статическим анализаторам и IDE. Из стабов извлекались: параметры конфигурации алгоритмов, доступные алгоритмы и типы возвращаемых результатов. Второй — репозиторий примеров `desbordante-core` [11], в котором для каждого примитива есть скрипты на Python с демонстрацией использования. Из примеров получались: тип принимаемого датасета, значения параметров по умолчанию и порядок вызова примитива.

2.1.1. `FD` — функциональные зависимости

$X \to Y$: значения столбцов X однозначно определяют значение столбца Y, примитив `FD` находит все такие зависимости в датасете.

Датасеты. Принимает один табличный датасет в виде кортежа `(путь, разделитель, has_header)`.

Алгоритмы. Доступно 12 алгоритмов: Aid, DFD, Depminer, EulerFD, FDep, FUN, FastFDs, FdMine, HyFD, PFDTane, Pyro, Tane. Алгоритм по умолчанию — HyFD.

Параметры. `table` (путь, разделитель, заголовок), `max_lhs`,

`is_null_equal_null`; у части алгоритмов — `threads`.

Результат. Метод `get_fds()` возвращает `list[FD]`. Каждый объект `FD` содержит `lhs_indices` — список индексов столбцов левой части, и `rhs_index` — индекс столбца правой части.

2.1.2. `AFD` — приближённые функциональные зависимости

`AFD` обобщает `FD`, допуская заданную долю нарушающих строк.

Датасет. Один табличный датасет.

Алгоритмы. Pyro (по умолчанию), Tane.

Параметры. `table`, `error` (порог ошибки, $\in [0, 1)$), `max_lhs`, `is_null_equal_null`; у Pyro — `threads`, `seed`.

Результат. `get_fds() -> list[FD]` (те же объекты, что в `FD`).

2.1.3. `FD_VERIFICATION` — верификация `FD`

Проверяет, выполняется ли заданная $X \to Y$ в датасете.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — FDVerifier.

Параметры. `table`, `lhs_indices`, `rhs_indices`, `is_null_equal_null`.

Результат. `fd_holds() -> bool`; при нарушении — `get_error() -> float` и `get_highlights() -> list[Highlight]`.

2.1.4. `AFD_VERIFICATION` — верификация `AFD`

Аналогично `FD_VERIFICATION`, но оценивает степень нарушения зависимости. В Desbordante `AFD_VERIFICATION` является псевдонимом `FD_VERIFICATION`.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — FDVerifier.

Параметры. Те же, что у `FD_VERIFICATION`.

Результат. `get_error() -> float` (доля нарушающих строк), `get_highlights() -> list[Highlight]`.

2.1.5. `AC` — алгебраические ограничения

`AC` описывает линейные соотношения вида $op(A, B) \in [l, r]$, где $op \in \{+, -, \times, /\}$.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — AcAlgorithm.

Параметры. `table`, `bin_operation`, `fuzziness` (доля допустимых исключений), `p_fuzz`, `bumps_limit`, `weight`, `ac_seed`, `iterations_limit`.

Результат. `get_ac_ranges() -> list[ACRanges]` (интервалы допустимых значений) и `get_ac_exceptions() -> list[ACException]` (нарушающие строки).

2.1.6. `ADC` — приближённые ограничения отрицания

`ADC` ищет ограничения, запрещающие определённые комбинации значений в парах строк, допуская заданную долю нарушений.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — FastADC.

Параметры. `table`, `evidence_threshold` (максимальная доля нарушений), `shard_length`, `minimum_shared_value`, `comparable_threshold`, `allow_cross_columns`.

Результат. `get_dcs() -> list[DC]`.

2.1.7. `AR` — ассоциативные правила

`AR` описывает правила вида $\{A\} \Rightarrow \{B\}$ с заданными поддержкой и достоверностью.

Датасет. Один транзакционный датасет в формате `tabular` (каждая строка — транзакция) или `singular` (две колонки: ID транзакции и элемент).

Алгоритмы. Единственный алгоритм — Apriori.

Параметры. `table`, `input_format`, `minconf`, `minsup`; для `singular` — `tid_column_index`, `item_column_index`, `has_tid`.

Результат. `get_ars() -> list[ARStrings]`, каждый с `left`, `right` (списки строк), `confidence`, `support`.

2.1.8. `DD` — дифференциальные зависимости

`DD` описывает связь между расстояниями в парах строк: если расстояния по столбцам X попадают в заданные диапазоны, то расстояние по Y ограничено сверху.

Датасет. Два датасета: основная таблица и таблица разностей (`difference_table`), задающая допустимые диапазоны для каждого столбца.

Алгоритмы. Единственный алгоритм — Split.

Параметры. `table`, `difference_table`, `num_rows`, `num_columns`.

Результат. `get_dds() -> list[DD]`.

2.1.9. `MD` — зависимости совпадения

`MD` описывает зависимость между схожестью строк двух таблиц: если схожесть по столбцам X превышает пороги LHS, то схожесть по столбцу Y превышает порог RHS.

Датасет. Два табличных датасета: `left_table` и `right_table` (могут совпадать).

Алгоритмы. Единственный алгоритм — HyMD.

Параметры. `left_table`, `right_table`, `column_matches` (список функций схожести), `min_support`, `max_cardinality`, `threads`.

Результат. `get_mds() -> list[MD]`, каждый с `lhs_bounds: list[float]` и `rhs: tuple[int, float]`.

2.1.10. `MFD_VERIFICATION` — верификация метрических `FD`

Проверяет, выполняется ли метрическая `FD`: если расстояние по X не превышает порога, то расстояние по Y не превышает δ.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — MetricVerifier.

Параметры. `table`, `lhs_indices`, `rhs_indices`, `metric` (`euclidean`, `levenshtein` или `cosine`), `parameter` (δ), `metric_algorithm` (`brute`, `approx` или `calipers`).

Результат. `mfd_holds() -> bool`; при нарушении — `get_highlights() -> list[list[Highlight]]`.

2.1.11. `NAR` — числовые ассоциативные правила

`NAR` обобщает `AR` на числовые данные: правило задаёт диапазоны значений антецедента и консеквента.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — DES.

Параметры. `table`, `minconf`, `minsup`, `population_size`, `max_fitness_evaluations`, `seed`.

Результат. `get_nars() -> list[NAR]`, каждый с `ante`, `cons` (`dict[int, ValueRange]`), `confidence`, `support`.

2.1.12. `PFD` — вероятностные функциональные зависимости

`PFD` обобщает `AFD`, используя вероятностную меру ошибки.

Датасет. Один табличный датасет.

Алгоритмы. Единственный алгоритм — PFDTane.

Параметры. `table`, `error`, `pfd_error_measure` (`per_tuple` или `per_value`), `max_lhs`, `is_null_equal_null`.

Результат. `get_fds() -> list[FD]` (те же объекты, что в `FD` и `AFD`).

2.1.13. Выводы

Из обзора примитивов следует несколько наблюдений, важных для проектирования системы. Во-первых, часть примитивов требует нескольких датасетов: `DD` принимает основную таблицу и таблицу разностей, `MD` — две таблицы для сравнения. Необходима поддержка произвольного числа датасетов на задачу.

Во-вторых, примитивы делятся по типу датасета: все, кроме `AR`, работают с табличными данными; `AR` — с транзакционными. Тип датасета

определяется примитивом и должен учитываться при создании задачи.

В-третьих, параметр `table` одинаков для всех табличных примитивов — это кортеж `(путь, разделитель, has_header)`. Аналогично, транзакционный датасет для `AR` характеризуется параметрами `input_format`, `has_tid`, `tid_column_index` и `item_column_index`. Все эти параметры описывают структуру файла и не изменяются без замены самих данных — то есть они являются свойствами датасета, а не задачи. Следовательно, их можно запрашивать у пользователя при загрузке датасета и хранить вместе с ним, не требуя повторного указания при каждом создании задачи.

В-четвёртых, ряд примитивов требует в конфигурации сведений о схеме конкретного датасета. Примитивы верификации принимают параметры `lhs_indices` и `rhs_indices` — индексы колонок проверяемой зависимости. `MD` требует параметр `column_matches` — список сопоставлений колонок левой и правой таблиц с выбором метрики сходства. Следовательно, при загрузке датасета целесообразно предоставлять клиенту информацию о его колонках, чтобы пользователь мог опираться на неё при создании задачи.

2.2. Существующая реализация

Цель данного подраздела — выявить места серверной части Desbordante, требующие доработки или переработки для соответствия требованиям, а также определить, в какие места в коде добавлять новую функциональность.

2.2.1. Стек технологий

Серверная часть веб-приложения Desbordante реализована в рамках предыдущих работ [9, 3]. Бэкенд построен по архитектуре web-queue-worker и старается придерживаться принципов чистой архитектуры [3].

Для реализации серверного API используется асинхронный веб-фреймворк FastAPI [8]. Метаданные хранятся в реляционной СУБД PostgreSQL [4]; для работы с базой данных из Python применяется биб-

лиотека SQLAlchemy [7]. Описание схем запросов и ответов, а также сериализация данных выполняются с помощью Pydantic [5]. Фоновые задачи профилирования выполняются воркерами распределенной очереди задач Celery [1] через брокер сообщений RabbitMQ [6].

2.2.2. Создание задачи

Задача профилирования создаётся через HTTP-запрос `POST /tasks/`. Передаются параметры: идентификатор датасета — через строку запроса (`dataset_id` как query-параметр), а конфигурация задачи — в теле запроса. Тип тела запроса — `OneOfTaskConfig`, аннотированный Pydantic-union из `FdTaskConfig` и `AfdTaskConfig` с дискриминатором по полю `primitive_name`. Такой подход позволяет Pydantic автоматически выбирать нужную схему валидации по значению поля.

Обработчик роута вызывает `CreateTaskUseCase`, который выполняет следующие шаги: проверяет существование датасета, создаёт запись `TaskModel` в базе данных со статусом `PENDING`, после чего ставит задачу в очередь Celery через `ProfilingTaskWorker.set`, передавая `task_id`, `dataset_id` и `config`. В ответ клиент получает схему `TaskSchema` — discriminated union по полю `status`, объединяющий `ProcessingTaskSchema`, `FailedTaskSchema` и `SuccessTaskSchema`.

2.2.3. Реализация примитивов

Каждый примитив представлен следующими сущностями: классом-задачей (наследником абстрактного `Task`) и Pydantic-моделями конфигурации и результата.

Абстрактный базовый класс `Task` реализует шаблонный метод `execute(table, task_config)`: из `task_config.config` извлекается конфигурация алгоритма, параметры сериализуются через `model_dump(exclude_unset=True)`, затем вызываются абстрактные методы `_match_algo_by_name()` и `_collect_result()`, а между ними — `algo.load_data(table=table)` и `algo.execute(**options)`.

Конкретный класс, например `FdTask(Task[FdAlgorithm, FdTaskConfig, FdTaskResult])`, реализует оба абстрактных метода: `_match_algo_by_name` возвращает нужный алгоритм через `match/case`, `_collect_result` — оборачивает результат `algo.get_fds()` в Pydantic-модели `FdAlgoResult` и `FdTaskResult`.

Конфигурация примитива представлена моделью `FdTaskConfig` с полем `primitive_name: Literal[PrimitiveName.fd]`, которое служит дискриминатором в `OneOfTaskConfig` — discriminated union всех конфигураций. Аналогично устроен `OneOfTaskResult` для результатов.

2.2.4. Выполнение задачи

Функция-воркер `profiling_task` получает из очереди три аргумента: `task_id`, `dataset_id` и `config`. Функция объявлена с базовым классом `ResourceIntensiveTask` — наследником `celery.Task`, который устанавливает лимиты ресурсов: временные (`time_limit`, `soft_time_limit`) задаются как атрибуты класса и обрабатываются Celery, а лимит памяти устанавливается через `resource.setrlimit(RLIMIT_AS, ...)`.

Логика профилирования реализована в `ProfileTaskUseCase`. По `dataset_id` загружается запись датасета из базы данных; поддерживаются только табличные датасеты. Содержимое файла скачивается из хранилища и читается с параметрами `separator` и `has_header` из модели датасета. Далее по значению `config.primitive_name` функция `match_task_by_primitive_name` возвращает экземпляр примитива — `FdTask` или `AfdTask`. Примитив выполняется вызовом `task.execute(table=df, task_config=config)`.

Обновление статуса задачи реализовано через сигналы Celery: `task_prerun` переводит задачу в статус `PROCESSING`; `task_postrun` — в `SUCCESS`, сохраняя результат; `task_failure` — в `FAILED`, сохраняя имя исключения, причину ошибки (`TaskFailureReason`) и трассировку стека. Вся эта информация хранится в `TaskModel`: поле `result` имеет тип `OneOfTaskResult | None` и хранится целиком в одном JSON-столбце

базы данных.

2.2.5. Получение статуса и результатов

Маршрут `GET /tasks/{task_id}/` возвращает запись задачи из базы данных целиком: статус, конфиг и поле `result` — весь результат профилирования в виде единого JSON-объекта без пагинации, фильтрации и сортировки.

2.2.6. Выводы

Существующая реализация поддерживает только два примитива — `FD` и `AFD` — и имеет ряд ограничений, препятствующих расширению функциональности.

Во-первых, задача жёстко привязана к одному табличному датасету; примитивы, работающие с несколькими датасетами (`DD`, `MD`), в текущей модели реализовать невозможно.

Во-вторых, при загрузке датасета не возвращается информация о его схеме. Эти данные необходимы пользователю для задания конфигурации таких примитивов, как `FD_VERIFICATION` или `MD`.

В-третьих, результаты профилирования возвращаются единым JSON-объектом без пагинации, фильтрации и сортировки, что неприемлемо для результатов с большим числом зависимостей.

Наконец, отсутствует маршрут получения списка задач — пользователь не может просмотреть историю своих запросов.

Рис. 1 иллюстрирует полный цикл работы с задачей в существующей реализации.

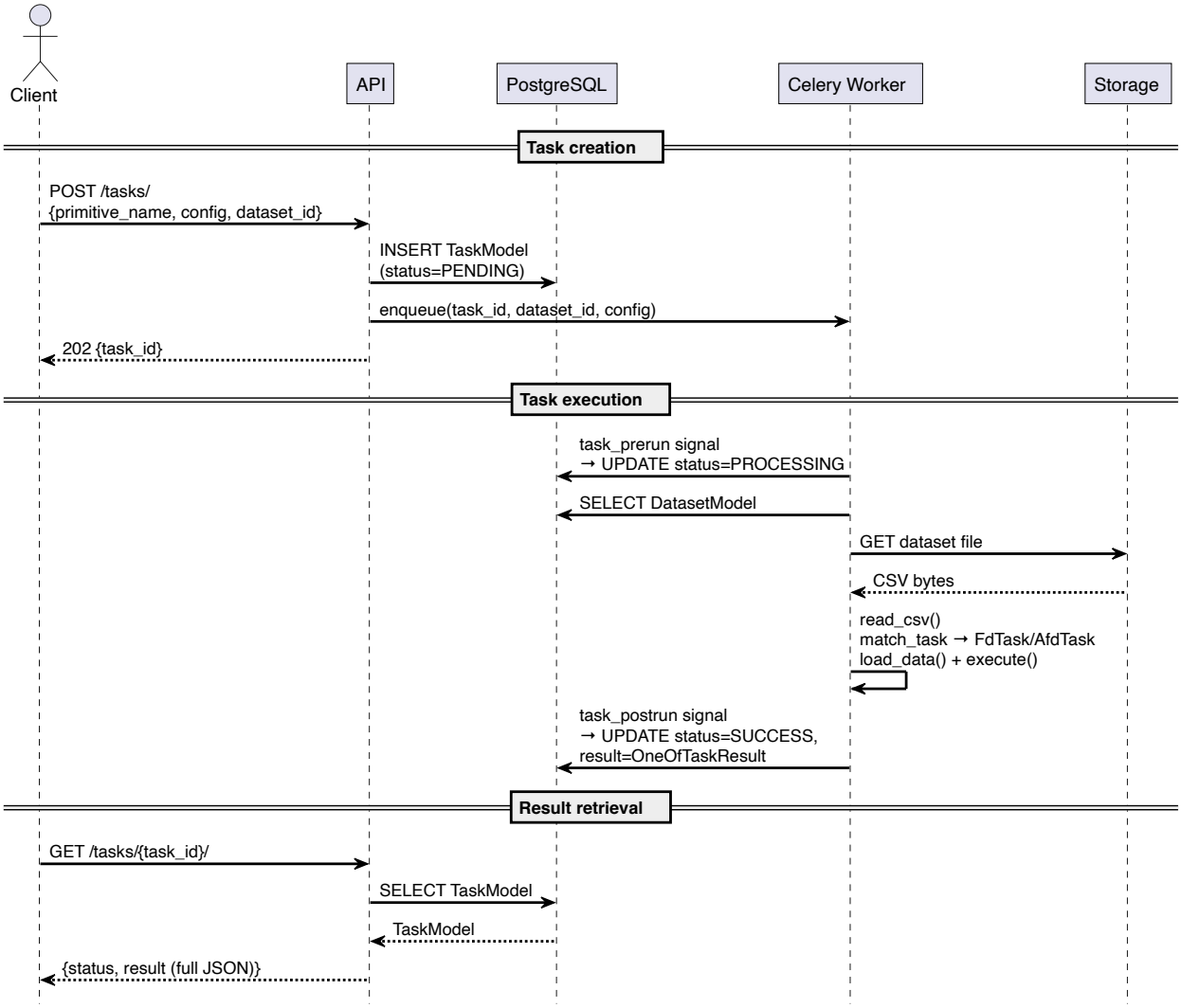

Рис. 1: Диаграмма последовательностей существующей реализации

2.3. Обновление статуса и результата задачи

Celery предоставляет три способа управления жизненным циклом задачи. Первый — сигналы: `task_prerun`, `task_postrun`, `task_failure`. С помощью параметра `.connect(sender=...)` обработчики можно привязать к конкретной функции-задаче, что исключает нежелательное срабатывание для других задач. Пока в системе только одна задача — `profiling_task` — такой подход вполне работоспособен. Однако при появлении второй задачи, например задачи предобработки датасета, общую логику обновления статуса придётся дублировать в отдельных обработчиках сигналов для каждой из них.

Второй способ — переопределение методов `on_success`, `on_failure` и `on_retry` в кастомном базовом классе — наследнике `celery.Task`. Общую логику (обновление статуса, сохранение результата или ошибки) можно вынести в один базовый класс, а конкретные задачи — унаследовать от него, переопределяя только то, что отличается. Это устраняет дублирование при добавлении новых типов задач.

Третий способ — использование готовых или реализация собственного result backend: архитектурно наиболее правильный подход, поскольку запись статуса и результата должна происходить именно в backend [1]. Однако он требует значительно большего объёма работы; кроме того, создание кастомных backend для Celery плохо задокументировано и требует изучения исходного кода библиотеки.

Выбор: изначально был выбран метод базового класса из-за простоты написания и использования. Однако в ходе обсуждения с другими разработчиками проекта было выявлено, что необходимо отлавливать исключение Celery `WorkerLostError`.

Данное исключение могут обработать сигналы Celery и result backend; методы базового класса вызываются на том же воркере, на котором происходит выполнение задачи и который теряется при возникновении `WorkerLostError`.

Поскольку необходимо управлять сохранением результата и ошибок задач (в том числе для пагинации результатов профилирования), было принято решение реализовать кастомный backend для хранения статусов и результатов в БД.

3. Описание решения

3.1. Модели базы данных

Для задач Celery и связанных сущностей была спроектирована иерархия моделей.

Базовый класс `BaseTaskModel[T]` — абстрактная модель с generic-параметром типа результата — добавляет поля `id`, `status`, `result` (тип `T | TaskErrorSchema | None`) и `finished_at`. Тип `result` извлекается из параметра наследования через `get_args(BaseTaskModel)` при объявлении класса.

Для хранения Pydantic-моделей в PostgreSQL был разработан кастомный SQLAlchemy `TypeDecorator` — класс `PydanticType`. При сохранении значение преобразуется через `jsonable_encoder` в словарь и сериализуется в `JSONB`. При чтении словарь передаётся в `TypeAdapter.validate_python` — discriminated union автоматически восстанавливается в нужный подтип.

`PreprocessingTaskModel` наследует `BaseTaskModel` и представляет задачу предобработки датасета: таблица `preprocessing_tasks` с полем `dataset_id` (уникальная связь один-к-одному). В `DatasetModel` добавлено поле `preprocessing` — отношение к `PreprocessingTaskModel`; статус и результат предобработки (метаданные датасета) хранятся в связанной записи, а не в самой модели датасета.

`ProfilingTaskModel` наследует `BaseTaskModel` и представляет задачу профилирования: таблица `profiling_tasks` с полями `params`, `owner_id`, `is_public`. Связь задачи с датасетами реализована через M2M-таблицу `ProfilingTaskDatasetLink`.

Результаты профилирования хранятся по двухуровневой схеме: `ProfilingTaskModel.result` содержит сводную статистику (или ошибку), а отдельные строки — в `ProfilingDepModel`, каждая запись хранит одну строку результата как JSONB. Разбиение на строки позволяет пагинировать, фильтровать и сортировать результаты средствами БД. Связи между моделями представлены на Рис. 2.

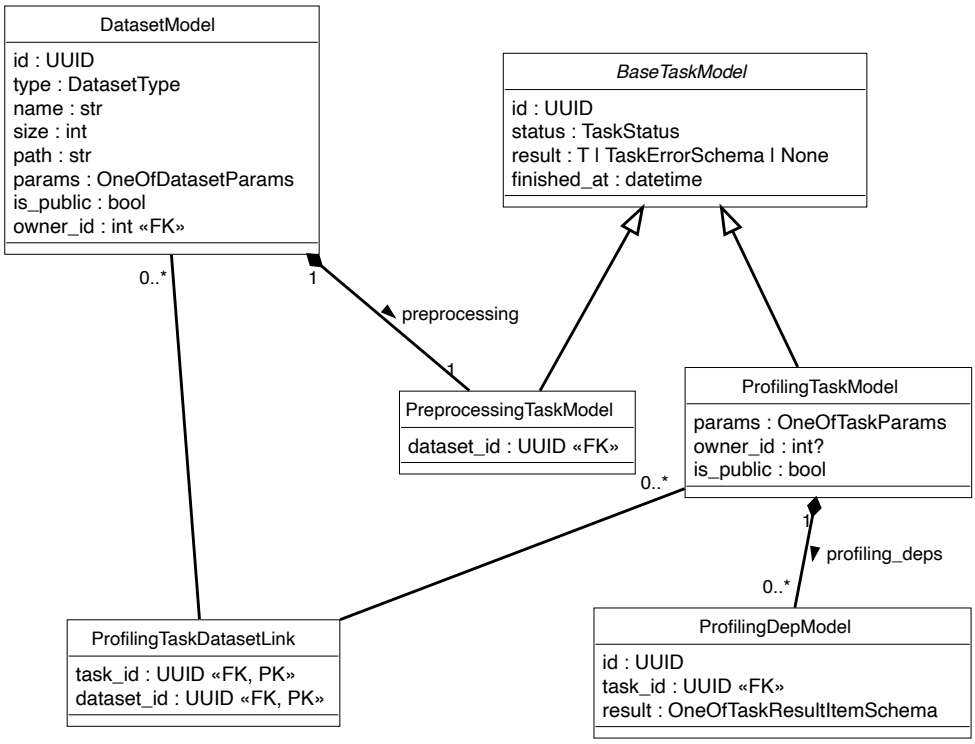

Рис. 2: Связи между моделями `ProfilingTaskModel`, `ProfilingDepModel`, `DatasetModel` и `ProfilingTaskDatasetLink`

3.2. Загрузка и предобработка датасетов

3.2.1. Параметры загрузки

Для каждого из трёх типов датасетов были спроектированы соответствующие классы параметров, описывающие структуру файла.

Были реализованы три класса параметров. `TabularDatasetParams` содержит `has_header` и `separator` (перечисление допустимых разделителей). `TransactionalDatasetParams` наследует его и добавляет `transactional_params` — вложенный discriminated union формата транзакций: построчный (`TabularTransactionalParams`) или двухстолбцовый (`SingularTransactionalParams`). `GraphDatasetParams` пустой, поскольку графовые датасеты в контексте данной работы не использовались. Все три объединены в discriminated union `OneOfDatasetParams` по полю `type` и хранятся в `DatasetModel.params` как JSONB.

3.2.2. Десериализация параметров

При реализации возникла проблема: формат `multipart/form-data` использующийся при загрузке датасета передаёт поле `params` как строку, тогда как Pydantic discriminated union (`OneOfUploadDatasetParams`) по умолчанию ожидает словарь и возвращает ошибку валидации. Для её решения был добавлен `model_validator(mode="before")` в базовый класс `BaseUploadDatasetParams`: если входное значение является строкой, оно десериализуется через `json.loads` до начала валидации.

3.2.3. Предобработка датасета

Для извлечения метаданных датасета была разработана Celery-задача `preprocess_dataset` с базовым классом `ResourceIntensiveTask` для ограничения ресурсов при выполнении задачи. Задача принимает единственный аргумент типа `DatasetForTaskSchema` с информацией для предобработки датасета и выполняет следующие шаги:

1. скачивает файл из объектного хранилища;
2. по типу датасета вызывает соответствующую функцию:
    - `get_tabular_info` — читает файл через `pandas.read_csv` и возвращает `TabularDatasetInfo`: количество строк, количество столбцов и список имён столбцов;
    - `get_transactional_info` — то же, дополнительно возвращает `unique_values` — список уникальных значений всех строковых столбцов;
    - `get_graph_info` — читает DOT-файл через `nx_pydot.read_dot` и возвращает `GraphDatasetInfo`: количество вершин, рёбер и флаг ориентированности.

Полученная информация предоставляется клиенту при запросе списка датасетов — в частности, `column_names` используются при создании задач с примитивами верификации, где пользователю необходимо указать индексы проверяемых столбцов.

3.3. Архитектура примитивов

3.3.1. Базовый класс примитива

Для унификации работы с примитивами был спроектирован новый обобщённый базовый класс `BasePrimitive[A, N, P, R]`, параметризованный четырьмя типами: класс алгоритма `A`, перечисление имён алгоритмов `N`, схема параметров задачи `P` и схема результата `R`.

Каждый конкретный примитив определяет три атрибута класса. `_algo_map` — словарь `Mapping[N, type[A]]`, отображающий имя алгоритма в его класс. `_params_schema_class` — класс схемы параметров, используемый при валидации. `allowed_dataset_type` — тип датасета, с которым работает примитив.

Базовый класс предоставляет два метода. `validate_params(cls, params)` выполняет валидацию через `TypeAdapter(_params_schema_class).validate_python(params)` и возвращает типизированный объект параметров. Абстрактный метод `execute(params)` реализуется в каждом конкретном примитиве: загружает данные в алгоритм, запускает выполнение и формирует результат.

3.3.2. Схема примитива

Для каждого примитива была введена унифицированная структура пакета `schemas/task_schemas/primitives/{name}/` из четырёх модулей:

- `algo_name.py` — `StrEnum` допустимых имён алгоритмов;
- `algo_config.py` — Pydantic-схемы конфигурации алгоритмов, объединённые в discriminated union `OneOf{Name}AlgoConfig` по полю `algo_name`;
- `task_params.py` — схема параметров задачи `{Name}TaskParams`, наследующая обобщённый `BaseTaskParams[P, C, D]` с полями `primitive_name`, `config` и `datasets`;

- `task_result.py` — `ResultItemSchema` (одна строка результата), `ResultSchema` (итог выполнения с `total_count`), схема фильтров `FiltersSchema` и перечисление полей сортировки `OrderingField`.

3.3.3. Реестр примитивов

Для связи примитивов по имени был введён словарь `primitives_map` в модуле `domain/task/constants.py` — отображение `PrimitiveName → класс примитива` для всех двенадцати реализованных примитивов. Добавление нового примитива в систему сводится к одной записи в этом словаре.

3.3.4. Задача профилирования

Была реализована Celery-задача `profiling_task` с базовым классом `ResourceIntensiveTask` и backend `ProfilingTaskBackend`. Задача принимает аргументы `params: OneOfTaskParams` и `datasets: list[DatasetForTaskSchema]`; `task_id` передаётся при постановке в очередь (`ProfilingTaskRunner.run`) и используется backend для привязки результата к записи в БД.

По имени примитива из `params.primitive_name` через `get_primitive_class_by_name` находится класс примитива. Для каждого датасета из `params.datasets` скачивается файл из хранилища (`download_dataset`), формируется словарь с готовыми данными (`DataFrame` или `nx.Graph`). Затем вызывается `primitive_class.validate_params` с конфигом, где `datasets` заменён на скачанные объекты — примитив получает данные, не зная о хранилище. После валидации инстанциируется примитив и вызывается `execute(params)`. Результат возвращается из задачи; сохранение в БД и создание записей `ProfilingDepModel` выполняет `ProfilingTaskBackend` при завершении. Полная последовательность создания и выполнения задачи представлена на Рис. 3.

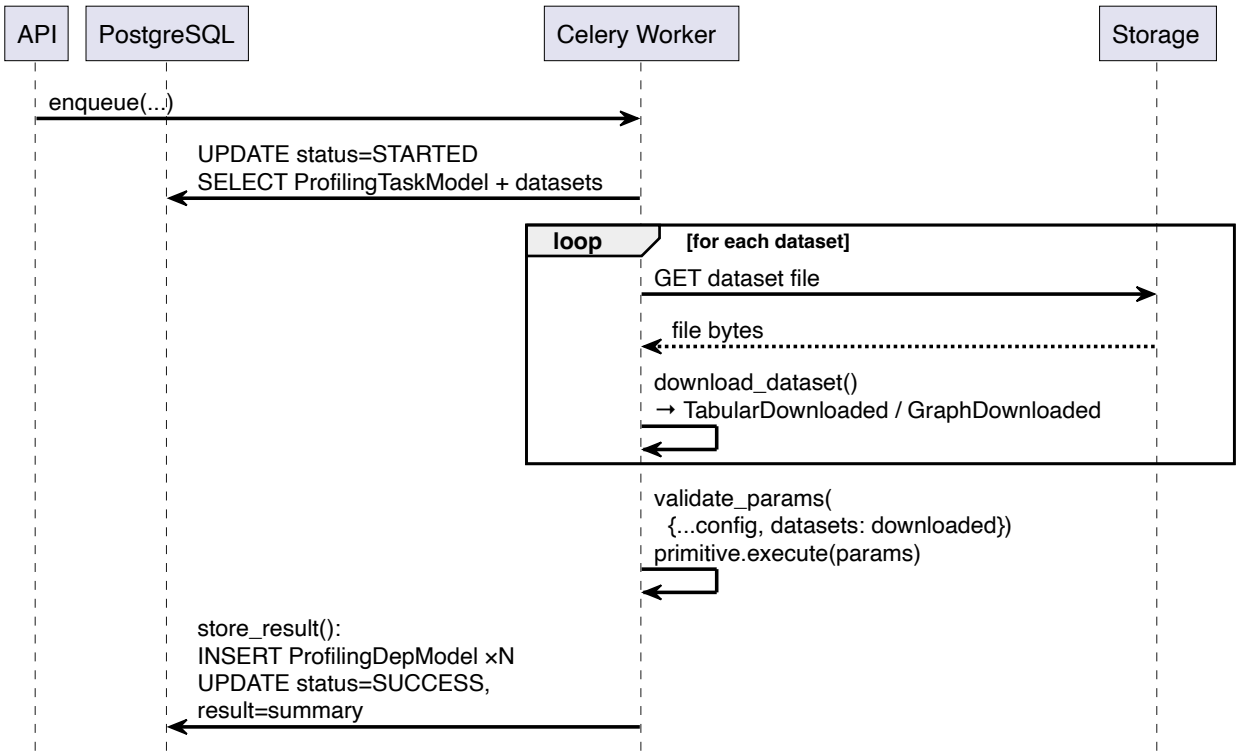

Рис. 3: Диаграмма последовательностей создания и выполнения задачи профилирования

3.4. Сохранение статуса и результатов задач

Был реализован базовый класс `BaseTaskBackend[T: BaseTaskModel]` — наследник `celery.backends.base.BaseBackend` для хранения результатов задач в БД. При реализации в качестве ориентира использовался `celery.backends.database.DatabaseBackend`. Как и `DatabaseBackend`, базовый класс применяет `SessionManager`, декоратор `retry` и контекст `session_cleanup` из `celery.backends.database` для работы с сессиями SQLAlchemy.

Класс переопределяет `_store_result`: получает метаданные результата через `_get_result_meta`, вызывает `_process_result` для преобразования в зависимости от статуса, обновляет запись в таблице БД. При ошибке исключение маппится в `TaskErrorSchema` с полем `reason` (`TIME_LIMIT_EXCEEDED`, `MEMORY_LIMIT_EXCEEDED`, `WORKER_LOST` и др.).

Наследники задают `task_cls` и при необходимости переопределяют `_process_success_result`. `PreprocessDatasetBackend` наследует `BaseTaskBackend[PreprocessingTaskModel]` и сохра-

няет результат предобработки как есть. `ProfilingTaskBackend` наследует `BaseTaskBackend[ProfilingTaskModel]` и переопределяет `_process_success_result`: из `result.items` создаёт записи `ProfilingDepModel`, а сводный `result.result` записывает в поле задачи. Связи между классами представлены на Рис. 4.

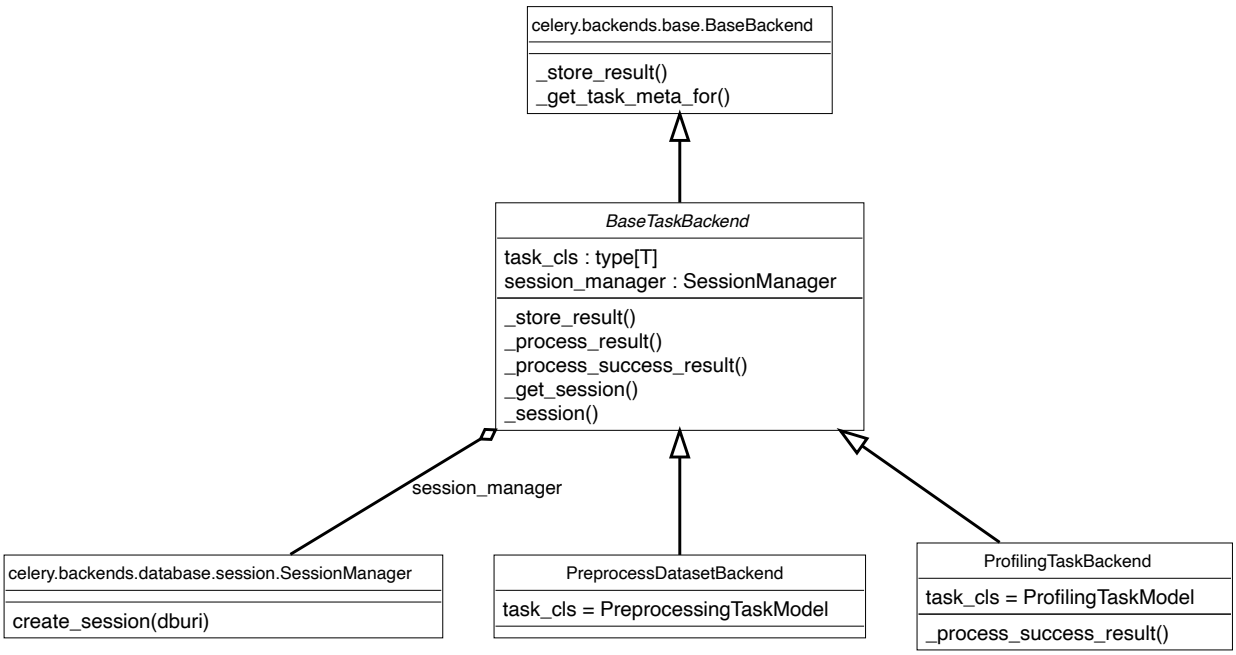

Рис. 4: Иерархия backend-классов: `BaseBackend`, `BaseTaskBackend` и `SessionManager`

3.5. Получение результатов

3.5.1. Пагинация, фильтрация и сортировка

Для поддержки постраничного вывода в `BaseCrud` был реализован метод `_get_many`, принимающий параметры пагинации, фильтрации и сортировки. Общее количество строк и сами строки получаются за один запрос к базе данных через оконную функцию `func.count().over()`.

Для получения результатов задачи был добавлен маршрут `GET /tasks/{task_id}/results/`. Обработку запроса выполняет `ProfilingDepCrud`, инициализируемый с именем примитива — по нему выбирается соответствующий `QueryHelper`. Для каждого примитива был написан `*QueryHelper` с тремя методами:

`make_filters` — формирует SQL-условия по переданным фильтрам, `get_ordering_field` — возвращает выражение для сортировки, `get_filtered_result_column` — возвращает выражение для модификации самого JSONB или `None`.

Если `get_filtered_result_column` возвращает не `None`, запрос строится с `CTE`: сначала вычисляется модифицированный столбец результата, затем к нему применяются пагинация и сортировка. Такая модификация потребовалась только для примитива `MD`: `JSONPath`-выражением из массива `lhs_items` убираются элементы с нулевым значением `boundary` или не совпадающей метрикой — частично отфильтрованный объект возвращается как новый JSONB.

Поскольку query-параметры приходят как строки, для фильтров типа `list[int]` или `list[str]` был добавлен `model_validator(mode="before")` в базовый класс `FiltersParamsSchema`: поля, значение которых начинается с `[`, десериализуются через `json.loads` до валидации. Полный процесс получения результатов показан на Рис. 5.

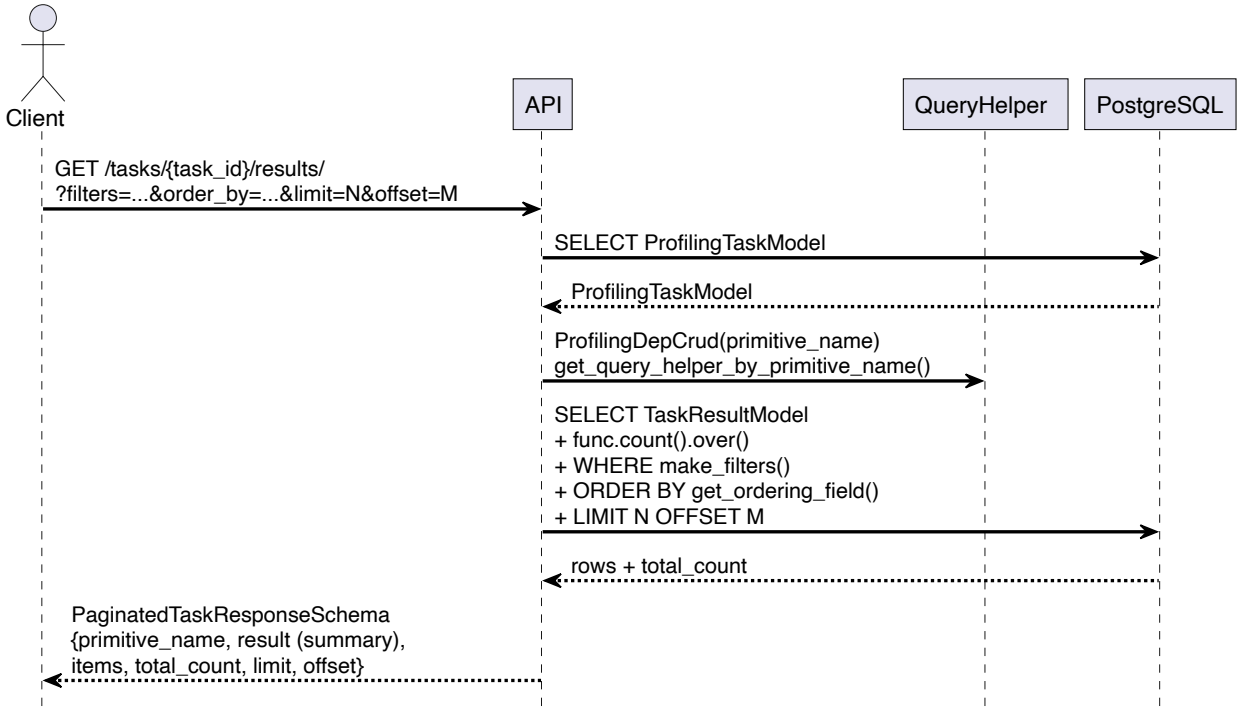

Рис. 5: Диаграмма последовательностей получения результатов задачи с фильтрацией, сортировкой и пагинацией

3.5.2. Список задач и датасетов

В существующей реализации возможность получить список задач пользователя отсутствовала. Были добавлены два маршрута, возвращающие пагинированные списки с поддержкой фильтрации и сортировки.

Маршрут `GET /me/tasks/` возвращает список задач текущего пользователя. Ответ типизирован как `PaginatedResult[TaskSchema]`, где `TaskSchema` — discriminated union по полю `status`: `ProcessingTaskSchema` (статус `PENDING` или `PROCESSING`, поле `result` отсутствует), `FailedTaskSchema` (статус `FAILED`, `result: TaskErrorSchema`) и `SuccessTaskSchema` (статус `SUCCESS`, `result: OneOfTaskResultSchema`). Поддерживается фильтрация по статусу и дате создания, сортировка по `status` и `created_at`.

Маршрут `GET /me/datasets/` возвращает список датасетов текущего пользователя в виде `PaginatedResult[PrivateDatasetSchema]`. Поддерживается фильтрация по типу, статусу, публичности, диапазону размера и дате создания, сортировка по имени, размеру и дате. Оба маршрута используют единый метод `BaseCrud._get_many`.

4. Развертывание и тестирование

4.1. Развертывание

Инфраструктура приложения управляется через репозиторий `desbordante-devops` [10] с помощью Kubernetes и ArgoCD. После пуша изменений в ветку репозитория серверной части Desbordante был запущен CI/CD-пайплайн: собран Docker-образ, после чего он был опубликован в реестре GitHub Container Registry. ArgoCD с настройкой `autoUpdate: true` автоматически обнаружил новый образ и развернул его в кластере без каких-либо изменений в конфигурации окружения. После завершения деплоя API стало доступно по HTTP.

4.2. Тестирование

Тестирование выполнялось на двух уровнях: ручное и автоматическое.

Ручное тестирование проводилось через API-клиент: для каждого из 12 примитивов были созданы датасет и задача с соответствующей конфигурацией, а затем получен результат профилирования. Для примитивов с несколькими датасетами (`DD`, `MD`) проверялась корректная привязка нескольких датасетов к одной задаче. Для примитивов верификации (`FD_VERIFICATION`, `AFD_VERIFICATION`, `MFD_VERIFICATION`) проверялось возвращение флага и соответствующих нарушений.

Автоматическое тестирование включает модульные и интеграционные тесты. Итоговое покрытие кода тестами составило 80%.

Реализованная функциональность соответствует требованиям, сформулированным в разделе 1.

5. Заключение

В результате работы была расширена функциональность системы профилирования датасетов в серверной части веб-приложения Desbordante. Были выполнены следующие задачи:

1. проведён обзор существующей реализации задач профилирования: архитектуры примитивов, цикла выполнения задачи, передачи данных и хранения результатов;
2. проведён обзор требуемых примитивов — типов датасетов, параметров алгоритмов и возвращаемых результатов;
3. спроектирована и реализована обновлённая система профилирования с поддержкой 12 примитивов: `FD`, `AFD`, `FD_VERIFICATION`, `AFD_VERIFICATION`, `AC`, `ADC`, `AR`, `DD`, `MD`, `MFD_VERIFICATION`, `NAR`, `PFD`;
4. реализованы хранение и получение результатов задач и списка задач с поддержкой постраничного вывода, фильтрации и сортировки;
5. выполнены развертывание и тестирование новой функциональности.

Ссылка на PR в GitHub-репозитории серверной части приложения: https://github.com/Desbordante/desbordante-server/pull/84 (имя пользователя — vladyoslav).

Список литературы

[1] Celery Documentation. — URL: https://docs.celeryproject.org/ (дата обращения: 10 марта 2026 г.).

[2] Desbordante: from benchmarking suite to high-performance science-intensive data profiler / George Chernishev, Michael Polyntsov, Anton Chizhov et al. // Proceedings of the 8th International Conference on Data Science and Management of Data (12th ACM IKDD CODS and 30th COMAD). — CODS-COMAD '24. — New York, NY, USA : Association for Computing Machinery, 2025. — P. 234–243. — URL: https://doi.org/10.1145/3703323.3703725.

[3] Nurmuhametov Rafik. — Backend architecture refactoring. — Desbordante, 2024. — spring. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Backend%20architecture%20refactoring%20-%20Rafik%20Nurmuhametov%20-%202024%20spring.pdf (дата обращения: 10 марта 2026 г.).

[4] PostgreSQL Documentation, PostgreSQL Global Development Group. — URL: https://www.postgresql.org/docs/ (дата обращения: 10 марта 2026 г.).

[5] Pydantic Documentation, Pydantic. — URL: https://docs.pydantic.dev/ (дата обращения: 10 марта 2026 г.).

[6] RabbitMQ Documentation, VMware. — URL: https://www.rabbitmq.com/documentation.html (дата обращения: 10 марта 2026 г.).

[7] SQLAlchemy Documentation, SQLAlchemy. — URL: https://docs.sqlalchemy.org/ (дата обращения: 10 марта 2026 г.).

[8] Tiangolo. FastAPI Documentation. — URL: https://fastapi.tiangolo.com (дата обращения: 10 марта 2026 г.).

[9] Vadim Yakshigulov. — Backend Reimplementation. — Desbordante, 2023. — spring. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Backend%20reimplementation%20-%20Yakshigulov%20Vadim%20-%202023%20spring.pdf (дата обращения: 10 марта 2026 г.).

[10] Vladislav Shalnev. — Automating infrastructure deployment with K8s. — Desbordante, 2024. — spring. — URL: https://github.com/Desbordante/desbordante-core/blob/main/docs/papers/Automating%20infrastructure%20deployment%20with%20K8s%20-%20Vladislav%20Shalnev%20-%202024%20spring.pdf (дата обращения: 10 марта 2026 г.).

[11] desbordante-core: Examples, Desbordante. — URL: https://github.com/Desbordante/desbordante-core/tree/main/examples (дата обращения: 10 марта 2026 г.).

[12] desbordante-stubs: Stubs for Desbordante, Desbordante. — URL: https://github.com/Desbordante/desbordante-stubs (дата обращения: 10 марта 2026 г.).